

MLX on M5+: Dense 27B and MoE acceleration

Research findings, revised design, and qualification kit

Prepared for Phil | Research cut: September 4, 2026

Decision: Build shared, state-correct inference components, but use separate dense and MoE execution policies. First qualify the existing indirect prototype and the shared Gated DeltaNet preparation path. Treat a packed-quantized paired-projection kernel as a controlled experiment, not the assumed winner.

Delivered now: A corrected routing benchmark, stronger output and noise gates, a config-driven workload/memory planner, reproducible host tests, and an independently reviewable hardening patch. The GPU kernel source is unchanged.

Validation boundary: 40 Python host tests passed, and the existing address/layout test passed 11,010,821 checks under AddressSanitizer and UndefinedBehaviorSanitizer. No new Metal compilation, M5 execution, GPU timing, or whole-model performance result was obtained. No branch or PR was published.

This report distinguishes **verified source facts**, **derived accounting**, **historical user measurements**, and **proposed experiments**. None of the projected allocation savings below is a measured throughput gain.

1. What deeper research changes

The earlier plan was too quick to treat familiar techniques as fresh opportunities. Small-batch quantized weight reuse already exists in MLX's merged `qmv_wide` implementation. The inspected MLX-LM code already compiles the GDN decay-gate expression and includes a packed recurrent kernel. Reimplementing those features would create a weak baseline, not a new contribution. [\[S06\]](#) [\[S08\]](#)

The broader search also found PMetal's M5 MPP dispatcher for two floating-point gate/up projections plus SwiGLU. Consequently, “fuse gate and up on M5” is not a defensible novelty claim. The narrower candidate here is an MLX-compatible implementation using existing packed quantized buffers, optional indirect activation rows, and a measured choice of tiling and epilogue. I did not establish that this exact combination is globally new. [\[S09\]](#)

State recovery has equally important prior art. ReplaySSM retains compact inputs instead of repeatedly storing full state. SGLang has merged a GDN accepted-prefix fold implementation, with an explicit operation-order contract and an all-layers entry point. We should adapt and credit that work rather than claim that journaling or a bulk commit was invented here. [\[S11\]](#) [\[S12\]](#)

TreeWY and Bole already address hybrid-model tree verification and state reconstruction. TreeWY's reported B200 evaluation disables prefix caching, uses greedy verification, and does not promise bit-identical token streams. Its concurrency-one table includes throughput below baseline, and wider trees are not yet a throughput win. Those limitations directly argue against making tree reconstruction the first low-latency M5 feature. Bole separately includes a complete-forward verification budget. [\[S13\]](#) [\[S15\]](#)

Finally, EVICT already treats MoE verification as a cost problem involving expert activation, not acceptance alone. An M5 policy should build on that principle while measuring this device and runtime. Published results on other accelerators are not estimates of our gain. [\[S14\]](#)

Revised priority

Priorit y	Work	Why it comes here
P0	Repair measurement and extract actual model geometry	Completed locally; prevents optimizing an unrealistic workload
P1	Qualify indirect routing and a bounded GDN preparation fusion	Concrete source-level opportunities; shared GDN work reaches dense and MoE
P2	No-repack quantized gate/up with separately tested epilogue	Potentially useful, but register and staging costs can erase savings
P3	Exact-order accepted-prefix state replay and verification policy	Important for committed-token cost; substantial prior art and correctness risk
P4	Tree verification, broader formats, cross-model automatic tuning	Only after a safe baseline and demonstrated application benefit

The order is a research decision, not a measured ranking. If profiling shows that GDN preparation is negligible, stop that branch and redirect effort. Layer count alone is not a time profile.

2. The actual dense and MoE workloads

“27B” is not an architecture. This plan uses the Qwen3.8-27B and Qwen3.6-27B text decoders, plus Qwen3.6-35B-A3B. Their inspected configurations give the following geometry; another MoE using MLA or another recurrent rule needs a separate plan. [\[S03\]](#) [\[S04\]](#) [\[S05\]](#)

Property	Dense Qwen 27B variants	Qwen3.6-35B-A3B
Decoder layers	64	40
GDN / full-attention layers	48 / 16	30 / 10
Hidden width	5,120	2,048
FFN intermediate width	17,408	512 per routed expert
Routed experts / top-k	Not applicable	256 / 8
Shared-expert intermediate	Not applicable	512
GDN key / value heads	16 / 48	16 / 32
GDN key and value dimensions	128 / 128	128 / 128
Full-attention KV heads / dimension	4 / 256	2 / 256

Three regimes, not one kernel contest

Apple's MLX study distinguishes compute-heavy prompt processing from bandwidth-constrained token generation on its tested M5 workloads. M5 GPU Neural Accelerators are accessed through the GPU programming stack; they are not the separate Apple Neural Engine. The M5 Max specification lists up to 128 GB memory and up to 614 GB/s bandwidth. These facts motivate measurement, but neither peak specification establishes delivered application throughput. [\[S01\]](#) [\[S02\]](#)

For **single-token dense decode**, a paired gate/up operation still reads two different weight matrices. It cannot halve those weight bytes. Reducing a few activation buffers may make only a small difference if weights dominate the critical path.

For **verification**, several input rows can amortize weight loading. The correct comparator includes qmv_wide, current split-K choices, and NAX routing, not a deliberately slow per-row loop. Sweep rows 1, 2, 4, 5, 6, 7, 8, 9, 12, 16, 17, 24, and 32; thresholds often matter more than a round-number benchmark. [\[S06\]](#) [\[S21\]](#)

For **MoE**, global batch size is insufficient. Record active expert count, the distribution of rows per expert, and sorted-run lengths. Eight tokens need not select the same experts. A kernel that relies

on weight reuse may see mostly one-row expert runs even though the overall batch has many routes.

For **prefill**, activation movement and intermediates deserve attention. But profile the full layer and model: a routing microbenchmark can improve while the complete computation stays unchanged or gets slower.

Memory accounting that changes the experiment

Assuming BF16/FP16 KV entries and FP32 recurrent state, the following are derived from the configurations, not observed memory measurements. They exclude weights, workspace, draft models, allocator overhead, vision processing, and fragmentation. [\[S03\]](#) [\[S04\]](#) [\[S05\]](#)

Per-request storage	Dense Qwen 27B	Qwen3.6-35B-A3B
Committed GDN state	144 MiB	60 MiB
Convolution history	2.8125 MiB	1.40625 MiB
Full-attention KV per token	64 KiB	20 KiB
KV at 131,072 tokens	8 GiB	2.5 GiB
KV at 131,072 tokens, four requests	32 GiB	10 GiB

The formulas are `linear_layers * value_heads * Dv * Dk * state_bytes` and `full_layers * 2 * kv_heads * head_dim * kv_bytes * context`. The planner reproduces these calculations from a local config.

Thus recurrent snapshots can matter, but long-context KV may consume far more memory. Optimizing only recurrent state is not a complete 128K serving solution. A hybrid prefix-cache entry must represent both kinds of state at the same logical token position.

3. Audit of the existing prototype

The original row-indirect prototype retains activations in token order and supplies a source-row map to the NAX projection loader. It avoids an expanded activation gather, but does not fuse gate/up, remove the down projection, or change the inverse sort. Its stated pilot covers affine 4/8-bit, not 6-bit or NVFP4. Native execution was never established. [\[S19\]](#)

Issues corrected in the delivered harness

Invalid representative routing. The benchmark used random expert IDs with replacement. Its skewed case drew eight route IDs from only two experts. That can be useful as an adversarial kernel test, but it is not an ordinary top-eight router workload. The corrected generator selects unique experts within each token, permits reuse across tokens, uses a deterministic seed, and records realized occupancy. Its weighted synthetic distribution is explicitly not a learned-router simulator.

Numeric equality labeled bitwise. A numeric equality comparison does not distinguish positive from negative zero. The revised comparison views floating payloads as equal-width unsigned integers. Native matched-path tests now use the same check. Host tests demonstrate signed-zero and NaN-payload distinctions; finite benchmark outputs remain required.

Incomplete comparison structure. Zipping output sequences could silently ignore an extra output. The revised helper requires identical nonzero output counts, shape, and dtype before comparison. It tests finiteness on all three paths, not only the candidate, and compares both JIT paths against the upstream tolerance reference.

Weak A/A acceptance. A mean near one can conceal an unstable confidence interval. A/A now requires the entire reported interval inside the declared tolerance, rejects invalid timing summaries, and retains the run-level drift check. Correctness is checked before and after timing.

The three benchmark arms remain upstream with one shared activation gather, matched contiguous JIT with that same gather, and indirect JIT without the expanded gather. This preserves the original causal comparison instead of charging the baseline for two gathers. The pair scope excludes route-plan construction; the switch scope includes routing and unsorting. Neither is a whole-model benchmark.

What remains deliberately unchanged

No `.metal` or kernel-header source was modified. The current prototype still has its original admission limits and device gate; no default-on optimization or global server hook was installed. Its warm, repeated-layer workload is not an uncached full-model workload.

The NAX implementation also inherits an inactive-SIMD-group guard. A current draft MLX revert reports mixed M3 Ultra/M5 Max JACCL failures associated with that optimization. That report does not prove this standalone prototype fails, but it warrants explicit native synchronization and mixed-device qualification before reuse. ^[S22]

A/A interval checks do not cure all benchmarking error. They cannot establish hardware representativeness, remove cherry-picking, or validate a kernel that has never compiled. The corrected benchmark is a prerequisite, not a performance result.

4. First shared feature: bounded GDN preparation fusion

The inspected Qwen GDN implementation constructs convolution input from old history and new projected QKV, saves the next history, runs depthwise convolution and SiLU, then normalizes and scales Q/K before recurrence. Q uses $\text{inv_scale}^{**2} * \text{rms_norm}$, while K uses $\text{inv_scale} * \text{rms_norm}$. The decay-gate expression is already compiled elsewhere. These exact boundaries, not a generic “fuse GDN” slogan, define the proposed experiment. ^{[S07] [S08]}

Proposed contract: Accept projected QKV, old convolution history, convolution weights, masks/valid lengths, and normalization constants. Return normalized Q and K, activated V, and the next convolution history. Keep the large recurrent matrix update and output projection separate in the first revision.

The initial kernel should read old history and new QKV through two source windows rather than constructing a concatenated tensor. A bounded per-head tile can compute convolution and activation, accumulate Q/K norm statistics, and publish the outputs and next history. Compare a two-stage version against a more fused version; repeated work across heads or excessive live registers could make the nominally smaller graph slower.

Compatibility requirements: Preserve convolution tap order, mask behavior, right-padded valid lengths, output dtype transitions, epsilon placement, and the existing Q scaling. Do not apply the scale again in the recurrence. Cover zero-length valid rows and cache-boundary transitions. Keep training and unsupported layouts on the original implementation until independently supported.

Why this belongs ahead of an oversized projection kernel: It is shared by the selected dense and MoE architectures and has a bounded interface that does not require new weight packing. That is a scope advantage, not proof that it dominates runtime. First capture its time contribution at single-token decode, verification rows, and chunked prefill. Reject it when its maximum possible Amdahl benefit is too small to justify maintenance.

Required experiments: Baseline graph; direct-window convolution/history only; convolution plus SiLU; then Q/K normalization. Hold each cumulative revision separately so a regression can be localized. Compare outputs and the next history, followed by at least one subsequent token. Count actual Metal dispatches from a capture instead of assuming every Python expression launches a kernel.

No GPU implementation of this new fusion was produced in this research pass. The supplied planner generates its shapes and coverage requirements.

5. Second shared feature: quantized paired projections without repacking

A generic fused SwiGLU is prior art. The useful engineering question is whether MLX's existing packed formats can support a paired projection across direct dense rows and indirect MoE rows without permanent concatenation or dequantized weight copies. Earlier MLX-LM gate/up fusion work explicitly backed away from its quantized cached-concatenation path after memory and prefill regressions. ^{[S09] [S10]}

Two kernel layouts must compete

Layout A: paired accumulators. Load an activation fragment, apply it to separate gate and up weight tiles, and retain both accumulator sets. This maximizes direct activation reuse but can increase register pressure. It is the simple reference candidate, not the default by assumption.

Layout B: projection-axis partitioning. Assign sibling SIMD groups to gate and up for the same output-column region. Each group retains one projection's accumulator. At the epilogue, exchange only the required result tile through bounded threadgroup storage and form SwiGLU. This avoids doubling the accumulator set inside each group but adds exchange traffic, barriers, and possibly more threadgroups for a given output width.

Both are proposed layouts, not tested kernels. Do not claim that threadgroup staging is always better than ordinary cached loads. Compare optional shared activation staging against direct

fragment loads, including the indirect source-row variant. Every cooperative operation and barrier needs a valid uniform execution contract.

Separate the savings and the numerics

First implement paired projections with separate outputs. Then add an activation epilogue. This distinguishes scheduling and activation reuse from the benefit of eliminating gate/up buffers. Preserve intermediate rounding boundaries during the first correctness comparison; an algebraically equivalent expression with different casts or transcendental instructions is not automatically the same numerical program.

For the dense geometry, two 4,096-token, 17,408-wide FP16/BF16 outputs occupy **272 MiB**. Avoiding both writes and their later reads would remove **544 MiB of logical intermediate traffic per layer**, excluding the final activation output and all other traffic. Those are derived upper-scope accounting values, not DRAM-counter measurements or peak-memory savings. Both distinct weight matrices are still read. ^[S04]

For MoE, attach the existing indirect row map only to the input projections. The down projection consumes newly computed per-route activations, so the same input-gather elimination cannot simply be applied again. Preserve router scores and the final expert reduction.

Precision and dispatch contract

Start with the pilot's verified packing assumptions, but require actual 6-bit coverage before claiming support for Phil's 6-bit target. Six-bit packing must use its real group/word layout; 32 // bits is not a correct general replacement. NVFP4 must retain its own scaling semantics rather than inherit affine dequantization.

Dispatch must distinguish single-row matvec, small-row weight-reuse kernels, and larger NAX tiles. The dense case uses actual projection dimensions; the MoE case also needs expert occupancy. Do not force NAX merely because the device has it. Keep the fast path opt-in until native results pass both correctness and whole-block gates. ^{[S06] [S21]}

Narrow research claim: a measured, no-repack, packed-quantized paired-projection design with interchangeable direct/indirect loaders and controlled epilogue rounding. Search did not establish global novelty. The result may be useful even if an equivalent layout is found elsewhere; attribution and a stronger baseline then replace a novelty claim.

6. Verification and state commit: optimize the whole round

A conventional chain with one correction/bonus token per round has expected committed length $1 + \sum(P(A \geq j), j=1..d)$, where A is the accepted draft-prefix length. The cost objective is that expectation divided by total draft, target verification, sampling, state-commit/recovery, and scheduling time. Verification input rows and emitted output-token counts are not interchangeable. This is an accounting model, not an empirical speed prediction.

Use observed acceptance-survival curves by workload, not $\text{depth} * \text{average_acceptance}$ as though every position were independent. For MoE, fit verification cost using measured expert occupancy and recent workload statistics. Future target-layer routes are not known for free before target

execution; any predictor must be identified as a predictor, and no policy may silently drop required experts. EVICT provides relevant prior art for adaptive verification. [\[S14\]](#)

Exact-order replay before closed-form trees

The proposed first state-recovery implementation should retain a committed checkpoint and a bounded journal of the inputs needed to replay accepted GDN updates. Rebuild only the accepted prefix using the verified recurrence order; do not invert a lossy state or re-run the entire projection stack. Reuse the checkpoint on zero acceptance. Materialize a normal committed state at cache-export boundaries.

This follows ReplaySSM and the inspected SGLang exact-fold implementation. That source explicitly preserves its original recurrence order and reduction configuration; it does not establish bitwise equivalence for a Metal port. An MLX port must match MLX's own operations, not substitute the SGLang normalization formula. [\[S11\]](#) [\[S12\]](#)

As a planning comparison, eight additional full dense GDN snapshots would occupy 1,152 MiB. Retaining eight positions of FP16/BF16 K/V plus FP32 scalar g/beta for the selected dense geometry is about 6.14 MiB. This smaller figure excludes convolution history, Q/output lifetimes, and workspace, and it still requires the 144 MiB committed state. It is not a measured allocation pattern of the current runtime. The planner labels the two alternatives explicitly. [\[S03\]](#)

The critical invariant is one logical committed position across KV, recurrent state, convolution history, masks, offsets, and sampler state. Test every rejection position, full acceptance, zero acceptance, cancellation, batch compaction, prefix export/import, and the next token. Tree layouts add parent/topology constraints and should be a later independent project.

Do not freeze the drafter choice

DFlash 2's first-party release includes a Qwen3.8-27B drafter and an M5 Max/oMLX demonstration. It is a relevant competing baseline, not evidence for our kernel or a transferable speed multiplier. Compare ordinary autoregressive decoding, the applicable native MTP path, and a qualified parallel drafter under identical target weights and sampling semantics. Include draft-model memory and its cache in the budget. [\[S16\]](#)

Distribution-preserving speculative decoding requires the correct proposal probabilities, acceptance rule, and rejection correction, not just high agreement on a few prompts. Choose depth from past or otherwise valid policy information, and preserve the chosen algorithm's correctness when changing schedules. Same-seed token identity is not in itself the definition of stochastic distributional equivalence. [\[S18\]](#)

7. Qualification that protects the result

Phil's historical August 17 M5 qualification is a useful failure case, not a verdict on today's implementation. Its serving-c4 row reports 88.2% acceptance while throughput falls from 49.38 to 29.06 tokens/s, TTFT worsens, and exact output identity fails. Another row improves raw throughput but still fails identity. This is why the promotion criterion must include correctness and complete workload behavior. [\[S20\]](#)

Four levels of evidence

- 1. Host and mathematical checks.** Test shapes, packing/address arithmetic, permutation identities, policy refusal, and reporting. These find defects but cannot validate Metal compilation or concurrency.
- 2. Native kernel qualification.** Compile the exact source and execute it on the target GPU. Check independent numerical references, matched-path payload bits where appropriate, poisoned padding, guards, and negative cases. Capture the selected kernel so a fallback cannot masquerade as a fast-path pass.
- 3. Whole-layer qualification.** Use actual dense and MoE shapes, all deployed quantization configurations, and both realistic and adversarial routing. Measure inputs, outputs, workspace, and live allocations. Repeat paired timing across independent sessions and record machine/compiler/runtime identity.
- 4. Serving qualification.** Measure committed output throughput, p50/p95 time per output token, TTFT, errors, cache hits, and physical memory behavior at c1 and c4. Exercise prefix hits and misses, mixed prompt/decode traffic, long prefixes, and cancellation. Do not combine ratios from incompatible baselines.

Timing and residency

Report synchronized wall time and, where available, GPU execution time separately. Keep compilation and one-time setup out of steady-state timing but report their cold-start cost. Pre-register a small shape matrix before tuning; use a held-out set and independent sessions for promotion, not only the best microbenchmark cell.

`mx.clear_cache()` clears MLX's memory cache, and `get_cache_memory()` describes unused memory retained from the system allocator. That is not a documented hardware-cache-flush API. A repeated single-layer benchmark can therefore have a residency pattern unlike a complete model. Use full-model runs or a declared rotating-weight workload; do not label allocator-cache clearing as cold GPU weights. ^[S17]

A/A checks must surround A/B comparisons. Retain raw samples, order, confidence intervals, and rejected runs. No automatic retrial-until-green rule. Thermals and interference remain possible even after local drift checks; several independent successful sessions are stronger evidence than thousands of repetitions in one session.

Proposed promotion thresholds

Gate	Initial engineering rule
Functional	All required output, state, cache and failure-path tests pass
Native engagement	Exact candidate binary/kernel confirmed on actual M5
Noise	A/A intervals and run drift inside declared bounds

Gate	Initial engineering rule
Performance	Held-out per-workload floor at least 0.95x; declared suite geomean at least 1.05x
Interactive latency	No unexplained TTFT or p95 output-latency regression
Memory	No persistent repacked weights; explicit live/physical headroom
Fallback	Unsupported shapes and failed qualification use known-good behavior

These thresholds are policy choices, not scientific constants. A specialized fast path can ship for a proven winning regime without forcing neutral or losing cases to use it. Report a confidence interval on the relevant aggregate; per-cell intervals alone do not establish the suite-level threshold.

For a hypothetical component occupying 20% of runtime, doubling its speed yields only $1 / (0.8 + 0.2 / 2) = 1.111x$ overall. This illustrates why isolated kernel multipliers should never be presented as whole-model improvements.

8. Memory and future-chip policy

Use a configurable budget below physical capacity, with separate allowance for target and draft weights, KV, committed recurrent state, temporary journals, live activations, and workspace. The supplied planner defaults to a conservative 90,000,000,000-byte planning budget; that is a chosen limit, not a hardware guarantee or a complete process-footprint estimate.

The planner never grants `safe_to_run`. Without caller-supplied resident-weight and workspace figures, even its limited budget comparison stays unknown. Its KV calculation does not model KV quantization. A later runtime governor must observe active allocations, allocator cache, actual process footprint, and system pressure rather than trusting this algebra alone.

New chips should not inherit performance policy from a name match such as “M5 or later.” Query the runtime's supported capabilities, compile and execute a bounded probe, and key qualifications by device, OS/compiler, runtime binary, kernel revision, dtype, format, and shape regime. Any of these changing invalidates the relevant result. An older non-NAX path is a comparator or fallback, not proof of future-chip suitability.

Keep inference optimization separate from model-quality changes. Reducing expert top-k, dropping experts, lowering target precision, or silently changing the sampling policy is not the same feature as accelerating identical computation. Such experiments need separate quality evaluation and must not contaminate a lossless baseline.

9. Implemented deliverable and next review units

The companion kit includes the revised indirect prototype, `benchmark-hardening.patch`, portable qualification helpers, and `planner/plan_workloads.py`. Example configs contain explicitly labeled extracted fields from the three official configs, not fabricated byte-for-byte originals. Generated manifests include dense, routed-expert, shared-expert, GDN, and vocabulary-projection shapes.

Executed locally	Result
New benchmark-helper host tests	20 passed
New workload-planner host tests	12 passed
Existing prototype policy/source tests	8 passed
Existing C++ address/layout test, ASan + UBSan	11,010,821 checks passed
Python byte compilation	Passed
GPU kernel/header comparison with original bundle	Unchanged
Native M5 qualification	Refused on Linux; not run

The host tests are not benchmark data. No dense projection kernel, GDN-preparation kernel, state-journal runtime, or automatic kernel selector was implemented in this pass. The concrete code improvement is trustworthy qualification infrastructure for those experiments.

Review unit A: benchmark hardening. Merge or apply independently of any optimization; it needs no performance claim. Preserve synthetic adversarial duplicate-route correctness tests while using valid top-k sampling for representative performance.

Review unit B: GDN preparation contract and native fixture. Freeze dtype, scaling, cache-length and padding behavior. Add the smallest direct-window fusion and evaluate its time fraction before expanding scope.

Review unit C: no-repack paired projection. Start with separate outputs and direct dense rows; compare accumulator layouts. Add indirect rows, then the activation epilogue, as separately gated increments. Do not expand format support by weakening checks.

Review unit D: accepted-prefix replay. Port the relevant prior art with attribution and explicit MLX operation order. Integrate complete hybrid prefix-cache semantics. Keep tree verification out of this unit.

Review unit E: policy and serving validation. Select only qualified regimes, include current drafters as competitors, and require c1/c4 results. The runtime must remain usable when speculation or a custom kernel is disabled.

Running the delivered tools

The planner is CPU-only and reads a local model configuration:

```
python planner/plan_workloads.py /path/to/model/config.json \
--output results/model-plan.json
```

After reviewing the manifest and memory headroom, the corrected native pilot can be tested on the M5. Start small; this example matches the routed-expert dimensions of the selected MoE, not a whole-model run:

```
cd indirect-prototype
./verify_m5.sh --tokens 512 --k 2048 --n 512 \
--experts 256 --top-k 8 --scope pair \
--output results/m5-moe-pair.json
```

Do not apply the hardening patch on top of the already-hardened copy. The patch is for the original prototype root. No tool here installs models, alters a server, or publishes a contribution.

10. Research limits and stopping rule

This research combined the supplied source bundles, pinned MLX/MLX-LM code, official model configurations, Apple documentation, peer project source, papers, and one historical user diagnostic. Search was targeted at direct overlap and disconfirming evidence, not a claim to have inspected every branch or private experiment.

The central uncertainty is still empirical: the chosen GPU kernel designs have not run on the target device here. Host address checks cannot establish cooperative-tensor behavior, occupancy, driver stability, thermal response, or performance. Paper and author-reported measurements retain their original device and workload scope.

The appropriate stopping rule is to reject a candidate that fails native correctness, changes required semantics, wins only against a weak baseline, or does not improve a useful complete workload. A useful outcome may be one small fast path, several rejected ideas, and a clearer bottleneck profile. That is stronger than a broad “M5 acceleration” claim without reproducible gains.

Bottom line: Keep the MoE indirect-loading experiment, broaden the work through shared GDN preparation and carefully scoped quantized projections, and make every decision in terms of correct committed-token latency and memory headroom. The delivered kit makes that research more reliable; it does not yet make the models faster.

Sources and provenance

[S01] Apple. *Apple debuts M5 Pro and M5 Max*. March 3, 2026. Official hardware specification and distinction between GPU accelerators and Neural Engine. [Open primary source](#).

[S02] Apple Machine Learning Research. *Exploring LLMs with MLX and the Neural Accelerators in the M5 GPU*. November 19, 2025. Device/workload-specific performance analysis; not evidence for these prototypes. [Open primary source](#).

[S03] Qwen. *Qwen3.8-27B config.json*. Official model repository, inspected September 4, 2026. Geometry input for derived accounting. [Open primary source](#).

[S04] Qwen. *Qwen3.6-27B config.json*. Official model repository, inspected September 4, 2026. [Open primary source](#).

[S05] Qwen. *Qwen3.6-35B-A3B config.json*. Official model repository, inspected September 4, 2026; includes shared-expert width. [Open primary source](#).

[S06] MLX PR #3764. *Add small-batch quantized matvec kernel (qmv_wide)*. Merged June 26, 2026. Prior implementation of small-batch weight reuse. [Open primary source](#).

[S07] MLX-LM. *mlx_lm/models/qwen3_5.py*, commit 32bb4e68791c941db382d6fc8fa5b35ba9f3d98b. Inspected GDN call path. [Open primary source](#).

[S08] MLX-LM. *mlx_lm/models/gated_delta.py*, same pinned commit. Compiled decay gates and packed recurrent implementation. [Open primary source](#).

[S09] PMetal. *mpp_fused_swiglu.rs*, commit 23c23dbabaedc12adaa92eda65736f6059f20097. Existing floating-point MPP paired projection/activation dispatcher. Its speed was not independently validated here. [Open primary source](#).

[S10] MLX-LM PR #1319. *Add optional fused SwitchGLU gate-up projection*. Closed without merge. Author-reported quantized cached-weight regression informs design risk. [Open primary source](#).

[S11] Dao AI Lab / NVIDIA. *ReplaySSM: Cache SSM Inputs, Not State*. First-party 2026 technical article. Relevant algorithmic prior art, not M5 timing evidence. [Open primary source](#).

[S12] SGLang PR #32692 and *gdn_replayssm_spec_fold.py*, merge commit f3fd8694947581e3fe6507a075a4b7ae62ff385b, July 31, 2026. Inspected exact-order fold source and state-tracking interface. [Open primary source](#).

[S13] Ghantasala. *TreeWY: Speculative Verification for Gated DeltaNet Hybrids*. arXiv:2608.20961v1, August 21, 2026. Read for mechanism and evaluation limitations. [Open primary source](#).

Sources and provenance (continued)

- [S14] *Making Every Verified Token Count: Adaptive Verification for MoE Speculative Decoding*. arXiv:2605.00342v1, May 2026. EVICT; prior art for verification-cost-aware scheduling. [Open primary source](#).
- [S15] Wang et al. *Bole: Efficient Tree Speculation for Hybrid-Attention Language Models*. arXiv:2608.01651v1, August 3, 2026. Abstract establishes direct overlap; not independently benchmarked here. [Open primary source](#).
- [S16] Inco AI. *DFlash 2: Keep Drafting Parallel*. August 18, 2026. First-party drafter release; relevant competing baseline. No published speed ratio is transferred to this proposal. [Open primary source](#).
- [S17] MLX API documentation. *clear_cache* and *get_cache_memory*. Inspected September 4, 2026. Allocator-memory cache semantics. [Open primary source](#). [clear_cache reference](#).
- [S18] Leviathan, Kalman and Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML 2023, PMLR 202. Distribution-preserving speculative inference foundation. [Open primary source](#).
- [S19] Supplied *mlx-nax-indirect-prototype.zip*, README, benchmark and kernels. Direct local audit. Original GPU status remains unvalidated; reviewed locally against the supplied copy.
- [S20] User's saved *Pasted markdown(20260817-170358).md*. Historical M5 native-MTP qualification; serving-c4 row. Private source retrieved from Library; not a current-run result or an upstream benchmark.
- [S21] MLX. *mlx/backend/metal/quantized.cpp*, commit b6368984b8e02a3fb3ee7986846c0fb85e1fccf7. Current baseline dispatch inspected in the supplied pinned source archive. [Open primary source](#).
- [S22] MLX PR #4404. *Revert inactive NAX MoE simdgroup skipping*. Open draft at inspection. Hardware-specific reported regression, not a demonstrated failure of this prototype. [Open primary source](#).